© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

This is the author's accepted version of a paper to be presented at the 2026 IEEE International Conference on Intelligent Transportation Systems (ITSC 2026), Naples, Italy.

Learning-Augmented Nonlinear MPC for Autonomous Racing

Arun Muraleedharan, Mubarak Abdu-Aguye, Ivo Simões, Michael Baumgartner, Danilo Caporale, Giovanni Pau
Autonomous Robotics Research Center, Technology Innovation Institute
Abu Dhabi, United Arab Emirates
arun.muraleedharan@tii.ae

***Abstract*—This paper presents a method for integrating Neural Networks (NNs) into a Nonlinear Model Predictive Control (NMPC) framework for autonomous racing, with the objective of enhancing model accuracy and thereby control performance. Our approach employs a data-driven NN trained on racetrack data to learn the modeling errors of traditional physics-based vehicle models. Unlike standard Model Predictive Control (MPC) formulations, which often require smooth and differentiable models, our method directly integrates the NN within a Randomized Model Predictive Control (RMPC) framework. RMPC, being a randomized optimization-based approach, does not impose continuity constraints on the NN function, enabling seamless integration without the need for explicit approximations or differentiability. This design allows for a more flexible and computationally efficient implementation, enhancing prediction accuracy and car performance in high-speed racing scenarios. Real-world race results demonstrate improved prediction accuracy, leading to more precise controls and faster lap times.**

***Index Terms*—Autonomous vehicles, model predictive control, neural networks, sampling based optimization, Graphics Processing Unit (GPU).**

I. Introduction

Autonomous racing serves as a rigorous testbed for high-speed multi-vehicle interaction, requiring the seamless integration of multiple coordinating modules. To succeed in such highly dynamic environments, these systems must operate at the vehicle's physical limits of handling while executing complex, real-time race strategies. In practice, this is achieved by two conceptual actors: a *planner*, which generates an optimal trajectory given a particular track and context, and a *controller*, which is responsible for executing control actions (e.g., steering, throttle, etc) to follow the trajectory. At its core, the controller's goal in autonomous racing is to maximize performance metrics (e.g., lap time, stability, and precision) while respecting physical constraints such as the track structure, actuation limits, etc. This requires controllers capable of handling non-linear dynamics, time-varying parameters, interactions with other vehicles and real-world challenges (e.g., sensor noise, latency, environmental changes, etc) in real time.

Model Predictive Control (MPC) has been widely adopted across multiple domains due to its ability to handle constraints and optimize control inputs over a finite horizon [1]–[5]. This latter ability arises through the use of an internal predictive model, which serves as a surrogate of the physical system being controlled. *Therefore, the effectiveness of MPC strongly relies on the accuracy of its predictive model*. In the majority of applications, the predictive models are derived from the principles of physics. They are therefore (largely) faithful to the physical realities of the system in question. Despite their provenance, the use of such models is fraught with several drawbacks. For instance, proper specification of such models is contingent upon full descriptive knowledge of the system's dynamics. This is often impractical for many problems, which negatively impacts the precision of the model and hence, the controller. Conversely, for all but the most simple systems, process dynamics are often mathematically dense due to the need to account for multiple interactions within and outside of the system, therefore incurring significant computational expense.



Fig. 1: EAV25 race car of Technology Innovation Institute (TII) at Abu Dhabi Autonomous Racing League (A2RL) 2025

MPC with physics-based models is widely used in autonomous racing. For instance, competitors of races such as Roborace, the Indy Autonomous Challenge, and A2RL [6] use different flavors of MPC [7]. Despite these successes, the development of advanced MPC controllers is limited by the challenges outlined above; overly simplistic models work but yield suboptimal performance, and very precise models (when available) are hard to identify and computationally infeasible due to their complexity.

NNs have gained traction as a means to improve prediction accuracy, especially where physics-based models fall short - as in the autonomous racing setting. This arises from their ability to learn complex relationships purely from data while being easily amenable to GPU-based acceleration, opening up new

avenues in the MPC domain. For instance, supervised NN-based approaches have been employed to approximate vehicle dynamics [8], [9] or directly map sensor inputs to control outputs [10], [11], highlighting the potential of data-driven techniques to complement traditional methods. Other approaches use neural networks to augment the MPC formulation to achieve targeted performance gains [12]. Another family of learning based approaches called Reinforcement Learning (RL), enables agents to learn optimal driving strategies solely through interaction with the environment. Simulation studies have demonstrated the integration of RL with model-based controllers to achieve better generalization and adaptability [13]–[15]. However, this class of methods are demonstrated only in simulations, and are yet to be applied to full-scale autonomous racing.

We summarize the major contributions of this work as follows:

- Implementation of a NN-augmented physical model inside a random sampling based MPC for autonomous driving at close to human speeds.
- Both the controller and the NN inference are executed in Graphics Processing Unit (GPU) realizing fast computation times.
- Achieves faster lap times and lower speed tracking errors using the improved model accuracy.

To the best of the authors' knowledge, this work provides the first experimental demonstration of neural network–based model error correction integrated within an RMPC framework for autonomous racing of full-sized vehicles. We provide a brief literature review in the proceeding section.

II. Related Work

As mentioned previously, data-driven methods have recently gained traction in the MPC space. These approaches can be broadly classified into three main categories, as follows:

A. Neural Networks for Prediction Modeling

As the performance of an MPC-based controller is heavily dependent on its system model accuracy, NNs can naturally be applied to the development or enhancement of such models directly from data. For instance, in [16], a NN is directly used to model the dynamics of a neutralization reactor in a pH control scheme via MPC. The network was trained on input-output pairs obtained from the real plant being controlled by a PI controller, and was shown to outperform the original PI-based control scheme and another approach utilizing an analytical model of the plant. Conversely, [17] consider an alternative approach where NNs (in this case feedforward and recurrent versions) are used instead to compensate for unmodeled dynamics given some nominal/analytical system model. The authors apply this enhancement scheme to four diverse control settings, and confirm the superiority of NNs to traditional MPC relying solely on analytical models.

Notably, NNs have also been used for prediction modeling in autonomous driving, as reported in [8]. The authors train a NN on control-state pairs of a car driving under complex conditions (e.g. unknown road-tire friction coefficients) and incorporate this into an MPC-based controller. The authors report that this approach outperforms a physics-based MPC solution which has full knowledge of the driving conditions.

B. Neural Networks for Control Policy Estimation

Another set of NN approaches have been developed to tackle the *computational expense* of solving the MPC problem. In particular, they attempt to learn a mapping from control inputs to optimal control actions directly, bypassing the actual (iterative) process of the MPC solution. For instance, [18] demonstrate that a NN can directly replace an MPC controller in an application setting with 252 states, 32 control inputs, and a receding horizon length of 140 timesteps. The authors show that the NN provides between 3-5$\mathbf{x}$ speedup, with less than 1% loss in closed-loop performance. Similarly, [19] applies a similar approach to building temperature control involving a model with 286 states and 42 disturbances. The resulting NN performs competitively with the full MPC formulation, while being almost 7$\mathbf{x}$ faster.

This approach has been applied to autonomous driving in [10], where the authors use a neural network-based MPC surrogate to control a vehicle chassis in real time, which was not previously possible via standard/online MPC.

C. Neural Networks in Control Law Formulation

Besides the system model and policy, the *cost function* used in the MPC formulation also affects performance [20]. This motivates a complementary usecase for NNs in the MPC framework; modulating the cost function parameters to enhance performance on the target task, therefore reducing reliance on the system model. This is explored in [21], where the authors use a NN to parameterize weight terms in a generic cost function. They show that by carefully selecting the network architecture and activations, operating constraints of MPC can be respected and therefore theoretical guarantees via existing analyses still hold. To explicitly account for safety under uncertain conditions, [22], [23] combine NN parameterization with Gaussian processes and Bayesian Optimization to handle uncertainty and ensure that the resulting MPC conforms to defined safety constraints.

In the context of autonomous driving, [12] used convolutional NNs to generate cost maps directly from images captured from a vehicle camera. These cost maps were then used together with an MPC controller to determine an optimal short-term driving trajectory / strategy. The authors show that their method achieved lap times competitive with the state of the art, supporting the efficacy of their approach.

III. Problem description

Autonomous racing involves multi-vehicle racing at the limits of handling in highly dynamic environments. Although the proposed framework applies to all racing scenarios, this study focuses on qualifying-style solo laps to clearly isolate and evaluate the impact of the proposed method.

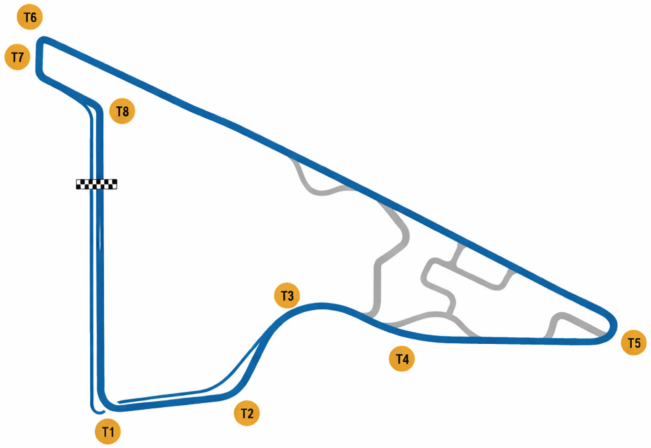


Fig. 2: Corners in Yas Marina North circuit, Abu Dhabi

The planner generates an optimal racing line as position and velocity references for the controller. At each time step, it publishes a trajectory segment from the ego vehicle's current state to a fixed prediction horizon, time-discretized into N_p reference points. The reference trajectory is defined as follows:

$$\begin{aligned} {}^{l}\chi_{ref} &= \{{}^{l}p_{ref,i}, {}^{l}v_{ref,i} | i = 0, 1, \cdots, N_p\}, \\ {}^{l}p_{ref,i} &= \{{}^{l}x^{p}_{ref,i}, {}^{l}y^{p}_{ref,i}, {}^{l}\theta_{ref,i}\}, \\ {}^{l}v_{ref,i} &= \{{}^{l}x^{v}_{ref,i}, {}^{l}y^{v}_{ref,i}, {}^{l}r_{ref,i}\}, \end{aligned} \tag{1}$$

where ${}^{l}\chi_{ref}$ is the reference path at time t that contains N_p path nodes. ${}^{l}p_{ref,i}$ denotes the relative position of the path node, ${}^{l}x^{p}_{ref,i}$ and ${}^{l}y^{p}_{ref,i}$, from the car's origin and ${}^{l}\theta_{ref,i}$ is the relative heading. ${}^{l}v_{ref,i}$ represents the velocity ${}^{l}x^{v}_{ref,i}$ and ${}^{l}y^{v}_{ref,i}$ at the respective point on the path, and ${}^{l}r_{ref,i}$ is the yaw rate.

The 2025 A2RL race used the north layout of the Yas Marina Formula 1 circuit. The layout of this circuit can be found in Fig. 2. The lap times are counted starting from the chequered flag in Fig. 2. Minimizing lap time requires optimizing trajectory, velocity, and control inputs under vehicle and track constraints [24]. While straights are typically driven at full throttle and largely limited by maximum power, lap time is primarily decided in corners. Cornering consists of braking, turn-in, apex, and exit phases, where the vehicle transitions from predominantly longitudinal force generation to combined-slip conditions and then to lateral-dominated motion before accelerating again. Because tire forces are inherently coupled, accurate longitudinal force prediction directly determines the available force envelope during combined-slip operation. Errors in longitudinal dynamics therefore propagate to yaw response, apex speed, and corner-exit traction, making corner performance—and ultimately lap time—highly sensitive to longitudinal model accuracy.

IV. Vehicle Model and Controller

The Emirates Autonomous Vehicle 25 (EAV25) vehicle used by TII Racing (Figure 1) for the A2RL 2025 race is based on the Dallara SF23 chassis. The car is fitted with a four-cylinder turbocharged engine and Yokohama Super Formula tires.

A. Curvilinear Single Track Model

A single track dynamic model [5], is used for the NMPC. As in [25], a curvilinear/Frenét coordinate frame is used to describe the state. Therefore, the global position and heading are not directly considered, but transformed to a state relative to the reference path.

*1) **Equations of Motion**:* The vehicle state and the input are given by

$$x = \begin{bmatrix} s & y_e & \theta_e & v_x & v_y & r & \delta & T & B & G \end{bmatrix}^T \tag{2}$$

$$u = \begin{bmatrix} \delta & T & B & G \end{bmatrix}^T \tag{3}$$

The variables s, y_e, and θ_e denote the progress along the path, the orthogonal deviation from the path, and the local heading, respectively. The longitudinal and lateral velocities $\dot{s}$ and $\dot{y}_e$ are considered, along with the yaw rate $\dot{\theta}_e$. The steering angle δ, throttle command T, brake command B, and gear command G represent the control inputs.

Thus, the equations of motion are

$$\begin{aligned} \dot{s} &= \frac{v_x \cos(\theta_e) - v_y \sin(\theta_e)}{1 - y_e \rho_{ref}(s)}, \\ \dot{y_e} &= v_x \sin(\theta_e) + v_y \cos(\theta_e), \\ \dot{\theta}_e &= r - \rho_{ref}(s)\dot{s}, \\ \dot{v}_x &= \frac{1}{m}\left(F_{x_r} - F_d - F_{y_f}\sin(\delta) + F_{x_f}\cos(\delta) + m v_y r\right), \\ \dot{v}_y &= \frac{1}{m}\left(F_{y_r} + F_{y_f}\cos(\delta) + F_{x_f}\sin(\delta) - m v_x r\right), \\ \dot{r} &= \frac{1}{I_z}\Big(l_f\big(F_{y_f}\cos(\delta) + F_{x_f}\sin(\delta)\big) - l_r F_{y_r}\Big), \end{aligned} \tag{4}$$

where $\rho_{ref}(s) = r_{ref} \,/x^{v}_{ref}$ is the curvature at the progress s, l_f and l_r are the distances from the center of gravity to the front and rear wheels, m is the mass and I_z the moment of inertia. (F_{y_f}, F_{y_r}) and (F_{x_f}, F_{x_r}) are the lateral and longitudinal tire forces at the front and rear wheels. Forces due to track banking and gradient are omitted for simplicity. F_d represents the aerodynamic force modeled as,

$$F_d = 0.5\,\rho_{air}\,A\,C_d\,v_x^2. \tag{5}$$

The air density ρ_{air}, the frontal area A and the drag coefficient C_d are used in (5).

In this work, we aim to correct the longitudinal velocity v_x. Its propagation is normally obtained as:

$$v_{x(t+1)} = v_{x(t)} + (\dot{v}_x \cdot dt) \tag{6}$$

To achieve our correction goal, we introduce an offset term Δv_x to cover effects unmodeled by the standard dynamics $\dot{v}_x \cdot dt$. Therefore, we propagate v_x as:

$$v_{x(t+1)} = v_{x(t)} + (\dot{v}_x \cdot dt) + \Delta v_{x(t+1)} \tag{7}$$

where $\Delta v_{x(t+1)}$ is obtained via a learned neural network. The particulars of this augmentation scheme are discussed subsequently in Section V.

*2) **Lateral Forces**:* The lateral forces generated by tires for front and rear axle f_{y_i}, $i \in [f, r]$, are modeled using a simplified Pacejka Magic Formula [26]. The usual macro-parameters B, C, D and E are used.

$$F_{y_i} = D_i \sin\Big(C_i \tan^{-1}(B_i\,\alpha_i) + \\ - E_i\big(B_i\,\alpha_i - \tan^{-1}(B_i\,\alpha_i)\big)\Big)\,, \tag{8}$$

where α_i, $i \in [f, r]$, are the slip angles.

The Pacejka coefficients B and D, which characterize the slope and peak of the tire lateral force–slip angle curve, vary with tire temperature. A dedicated grip estimation module updates these parameters for each tire using internal and surface temperature measurements. This enables accurate lateral tracking and enforces conservative operation when the tires are outside their optimal thermal range.

*3) **Longitudinal Forces**:* Considering a rear wheels drive layout, the axle longitudinal forces are modeled as

$$\begin{aligned} F_{x_f} &= -C_{b_f} B - C_{ro_f}\,, \\ F_{x_r} &= F_{engine}(T, v_x, G) - C_{b_r} B - C_{ro_r}\,, \end{aligned} \tag{9}$$

where C_{ro_i} is the rolling resistance at axle $i \in [f, r]$. The braking force is represented as $C_{b_i} B$, with C_{b_i} the maximum brake force and $B \in [0, 1]$. $F_{engine}(T, v_x, G)$ is a nonlinear

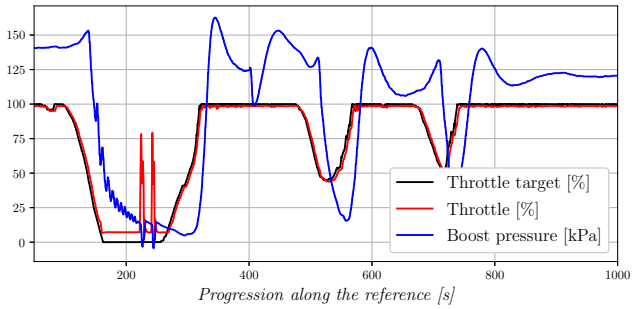


Fig. 3: Throttle and boost pressure

mapping that describes the drive force generated at the rear wheels as a function of throttle input, engine speed, and gear. For the turbocharged engine used in EAV25, this relationship is strongly nonlinear over the operating range. The model in (9) is identified from engine bench data and subsequently refined using vehicle test data.

The identification assumes that boost pressure has reached its target value, which reduces accuracy during boost transients. As shown in Fig. 3, the boost pressure continues to rise after the throttle stabilizes (around $s = 300$) and exhibits a delayed drop with an intermediate spike when the throttle is lifted (around $s = 100$). These unmodeled dynamics introduce errors in longitudinal force and speed prediction within the controller. The proposed neural network compensates for these effects and improves speed prediction under transient conditions.

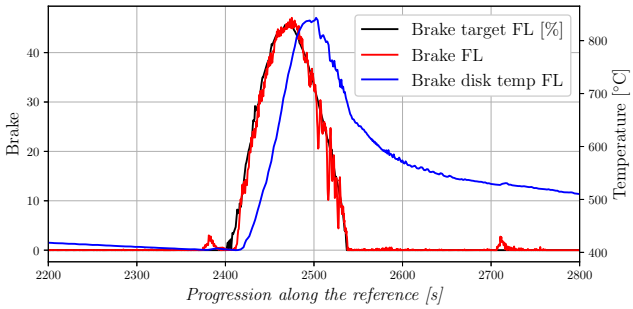


Fig. 4: Brake and disk temperature

The vehicle is equipped with Brembo carbon-ceramic brakes, whose maximum brake force C_{b_f} and C_{b_r} varies significantly between cold conditions and the optimal operating range of 400 °C–900 °C. Brake disc temperature sensors are used to parameterize C_{b_f} and C_{b_r}as functions of measured temperature. During heavy braking (e.g., around $s = 2400$ in Fig. 4), disc temperature rises rapidly. The control stack assumes the temperature at brake onset to determine C_{b_f} and C_{b_r}, which introduces errors under steep thermal transients. Additionally, hydraulic pressure build-up delays are not modeled in the MPC. These unmodeled thermal and actuation dynamics contribute to longitudinal prediction errors, which are mitigated by the proposed neural network.

The gear shift control is handled by a separate logic that request a shift to the low level when reaching the desired shift conditions. Gear G in the prediction horizon is propagated following the same shift logic that is in the module.

B. Controller

The controller employed in this study is a MPC framework solved using a randomized optimization approach. To accurately capture the vehicle behavior, a dynamic bicycle model (4) is utilized. To ensure the vehicle operates close to, yet within, its stability limits, the proposed framework enforces appropriate input and safety constraints (Section IV-B1).

For reliable real-time performance, a sample-based approach is chosen to solve the nonlinear optimization problem. The sample-based method used in this work is known as RMPC, and is attracting increased attention recently [27]–[31]. The readers are referred to [5] for the details of the proposed solver scheme.

The previous works on RMPC had no mechanisms of model error correction. This work improves on the model accuracy using a NN trained on track data. Another notable contribution is that the proposed RMPC and NN are executed in GPU for realtime control at 100Hz.

*1) **Input and safety constraints**:* Two types of constraints are used in this formulation. The first are the hard constraints that ensure the feasibility of input commands. The range of control inputs u and their rate of change $\dot{u}$ are enforced during sample generation to respect physical limits of actuation systems and to prevent jerky inputs. The second type are the soft

constraints that keeps the vehicle within it's dynamic limits. During high-speed operation, vehicle stability is maintained by ensuring that the tires do not exceed their available grip limits. The maximum resultant force at the tire contact patch is regulated through a soft constraint incorporated into the cost function. The combined tire force is constrained as follows.

$$F_i = \sqrt{(F_{y_i})^2 + (F_{x_i})^2} \leq \mu_i F_{z_i}, \quad \forall i \in \{f, r\}. \tag{10}$$

F_i represents the total force at the tire i. Forces F_{x_i}, F_{y_i} and F_{z_i} are the forces along x, y and z directions, respectively. Tyre index i can be f or r representing front and rear axles. μ represents the friction coefficient between the tire on the road.

*2) **Cost function**:* The controller aims to track the reference trajectory χ^{ref} with high precision while operating within the physical limits of the vehicle. However, these objectives are inherently coupled and may become conflicting in practice. This compromise is managed by embedding a tire grip constraint (10) within the cost function as a steeply increasing cost. By appropriately weighting the quadratic costs on tracking errors along with the grip cost and other cost components, a balance is achieved between trajectory tracking performance and compliance with stability limits. Additionally, regularization terms on the control inputs are included to promote smooth actuation. Additional costs penalizes when the position of the car goes beyond track bound [5]. The reader can refer to [32] for the path tracking performance of this dynamic model and [29] for the weight parameter matrices.

*3) **Formulation of input optimization problem**:* The optimization problem to be solved in every control step in order to generate optimum control input series can be formulated as follows:

given: $x(0|t) = x(t),\ \chi^{ref}$,
find: $u(k|t),\ (k \in \{0, 1, \cdots, N-1\})$
which minimize: $J(u_t = \{u(k|t)\}),\ (k \in \{0, 1, \cdots, N-1\})$
subject to: Input constraints (IV-B1),
Grip limit constraints (10),
Car dynamics (4).

V. Neural Network Design and Training

As discussed previously (Section IV-A), the prediction of the vehicle's longitudinal speed depends on multiple systems which have complex dynamics. For instance, the engine model specifications provide only steady-state peak torque values as functions of engine speed and throttle position. However, throttle application results in a delayed torque response due to turbocharger boost dynamics. Similarly, the aerodynamic drag coefficient is currently assumed to be constant, while in reality, it varies with vehicle ride height among other factors. The generation of brake force is also sensitive to delays in pressure accumulation within the hydraulic system, and bounded by brake disk temperatures which vary steeply. These effects are not included in the current dynamic model used within the MPC framework, which severely affect model accuracy and by extension, the resulting/downstream speed predictions.

To compensate for these unmodeled effects, we propose the use of a Long Short-Term Memory (LSTM)-based neural network [33] to additively correct the outputs of the base dynamic model, as in (7). The LSTM is a type of *recurrent* neural network i.e., it is temporally-aware and accepts sequences as input, which makes it a suitable candidate for capturing the delayed torque response and other temporal effects. In other words, this means that the offset term $\Delta v_{x(t+1)}$ in (7) is obtained from the neural network $g(.)$ i.e., $\Delta v_{x(t+1)} = g(x_{(t-h:t)})$ where h is some temporal window/horizon size, indicating how many historical samples are considered for the current prediction.

The training samples for the LSTM are generated as follows. From data collected in previous runs/races, the values of the car's state x (i.e., steering, brake, gear, etc as in (2), augmented with track curvature information) are isolated, and the corresponding errors between the car's actual velocity (as measured by onboard sensors) and the longitudinal model's predictions are computed. This is performed in overlapping/sliding-windows with width h. The model structure is shown in Figure 5, and shows that the network aims to regress the longitudinal velocity error from a sequence of the car's states $x_{(t-h:t)}$. Note also that in practice, the model regresses a fixed number (N) of error estimates, and these N estimates are linearly extrapolated to match the number of timesteps in the controller's receding horizon i.e., the extrapolated vector of error corrections correct *all* predictions from the base longitudinal dynamics model all at once. This is done for computational expediency, due to the tight timing constraints on the control loop.

Fitting a neural network to predict the errors in v_x alone might be unstable since such a model will be unaware of the broader physics underlying the car's motion. To avoid this, the LSTM is trained in a *multitask* manner. That is, the model is trained to compute the errors not only in v_x, but in v_y and r as well. Therefore, the model's actual training cost/loss function is the sum of prediction errors (i.e., mean squared error) across v_x, v_y and r. Then at runtime, predicted values of v_x within the RMPC are corrected using the trained LSTM, improving the longitudinal speed estimates. Note that error predictions for v_y and r are not used in this work, and remain available for use for future applications.

The network design and training are implemented using the PyTorch deep learning framework [34]. Training is performed on an RTX2000 GPU for 200 epochs with a batch size of 32, with network weights optimized using an Adam optimizer with a learning rate of 0.001. Once training is completed, the model is exported to TorchScript format for GPU-capable deployment and compatibility with the RMPC implementation in the car's software stack.

VI. Results

The controller is designed to follow the trajectory provided by the planner as closely as possible while ensuring the vehicle remains within its dynamic limits. The plots in this section compare two laps from the same day. One, the fastest lap with the model corrected with proposed NN and the fastest

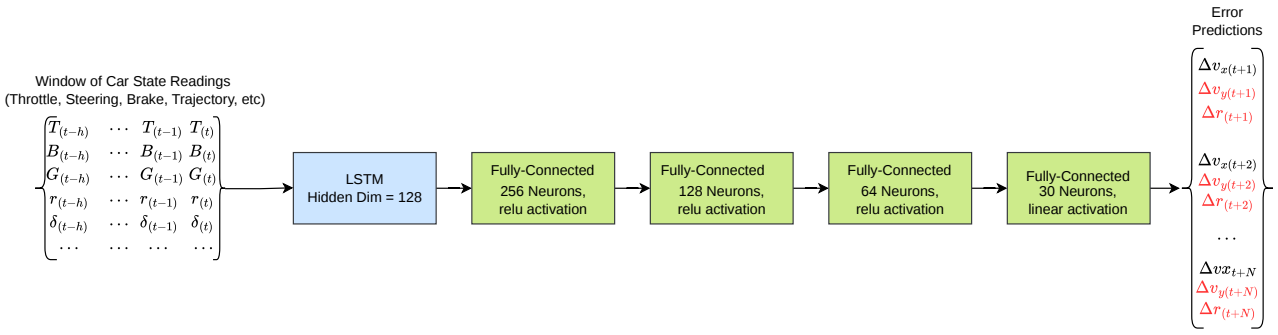


Fig. 5: Neural Network Overview. Outputs in red are considered during network training, but discarded at inference time.

lap without the NN enhancement. Data from the back-to-back sessions are chosen to avoid other factors affecting the comparison. On the x-axis, s represents the progress along the reference racing line.

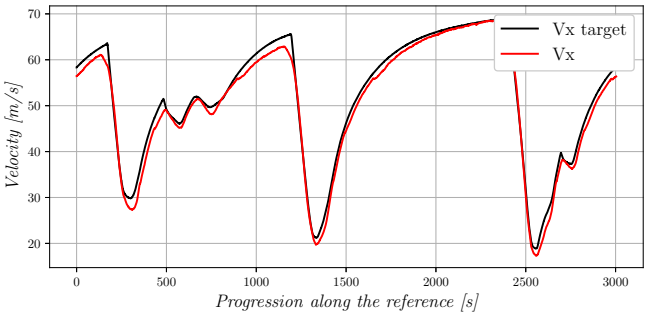
Fig.6 shows the speed target tracking performance of the controller. The reference trajectory is published by the online speed planner for the given grip level. The controller is able to stay much closer to the reference when the NN corrections are applied to the predictions. This is particularly visible at corner exits ($s = 300, 1300$ and 2600), where the delay in boost pressure causes an overestimate of torque and hence, delay in acceleration. The MPC corrected by NN is performing better, showing lower and closer to zero speed error as seen in Fig.6(c).

Focusing on the sector 1, Fig.7 shows a closer look at the speed tracking performance. The speed tracking error is consistently lower by 1m/s with the corrections at s from 250 to 850. At $s = 700$, Fig.7(c) shows that the throttle lift in case of MPC without correction is unnecessarily higher. While the proposed MPC with correction anticipates throttle, being aware of boost delays using the LSTM. The laps with and w/o corrections came up to lap times of 64.9s and 65.4s respectively.

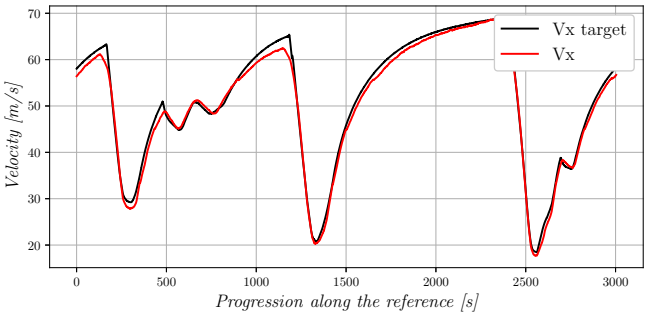
VII. Conclusion

The proposed RMPC framework is shown to achieve improved speed predictions and tracking while operating a Super Formula car at the limits of handling on a Formula One circuit. By integrating the proposed NN-based corrections, the controller effectively captures unmodeled dynamics in the powertrain and braking systems, including turbocharger behavior and actuator delays. This enhancement enables improved performance under high-speed, limit-handling conditions outperforming the baseline.

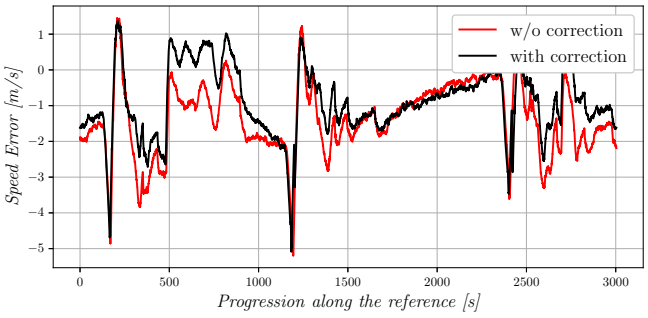
The on-board computer is a Neousys RGS-8805GC with an AMD EPYC™ 7003 series "MILAN" CPU and NVIDIA® RTX A6000 GPU. The baseline controller achieves a computational time of 4ms on the given computer, addition of the proposed NN increases the computation time by 1ms. A control frequency of 100 Hz is employed, leveraging the fast computation enabled by the GPU.



(a) Velocity tracking w/o correction



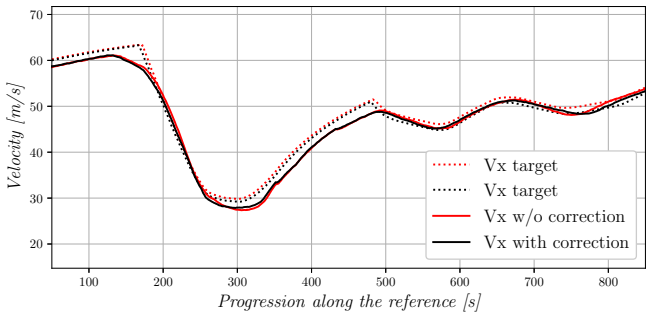
(b) Velocity tracking with correction



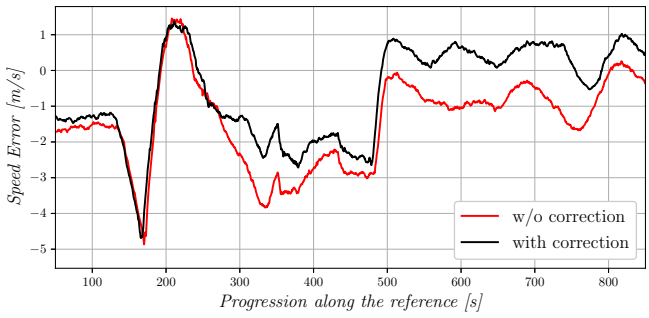
(c) Speed error

Fig. 6: Speed tracking

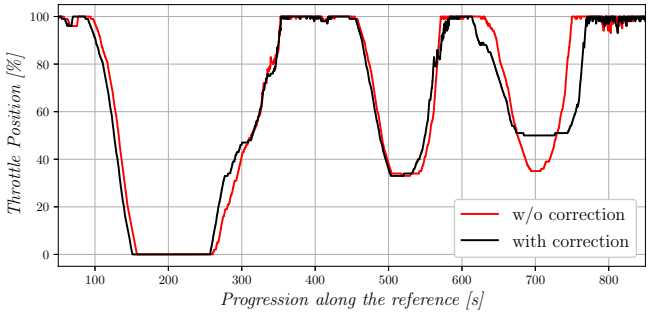
Future extensions of this work will explore the integration of neural networks for modeling vehicle lateral dynamics. Given



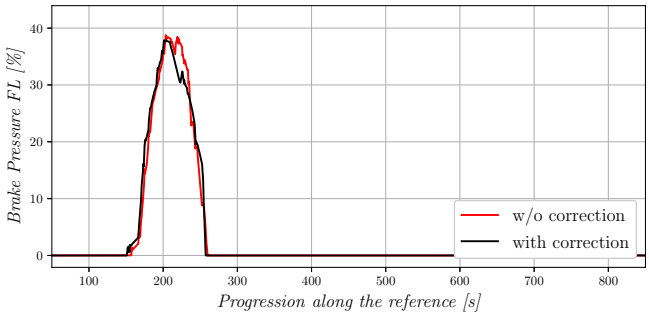
(a) Longitudinal velocity



(b) Speed error



(c) Throttle



(d) Brake pressure

Fig. 7: Sector1: speed tracking

the highly nonlinear nature of tire behavior and the inherent limitations of physics-based tire models, neural network–based models are expected to provide substantial improvements in lateral force accuracy, improving controller performance.

ACKNOWLEDGMENT

The authors would like to thank the TII Racing team members — Alberto Di Girolamo, Alessandro Palermo, Alessandro Tonoli, Andrea Finazzi, Andrea Vergine, Davide Malatesta, Ginevra Cerri, Luca De Mori, Matteo Scucchia, and Omar Al Ali — for their invaluable support in preparing the testing platform and developing the software framework used in this work.

Special thanks to A2RL organizers for the exciting championship and fast, reliable race cars.

REFERENCES

[1] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Transactions on control systems technology*, vol. 15, no. 3, pp. 566–580, 2007.

[2] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, 2022.

[3] A. Raji, D. Caporale, F. Gatti, A. Toschi, N. Musiu, M. Verucchi, F. Prignoli, D. Malatesta, A. F. Jesus, A. Finazzi *et al.*, "er. autopilot 1.1: A software stack for autonomous racing on oval and road course tracks," *IEEE Transactions on Field Robotics*, 2024.

[4] A. Wischnewski, T. Herrmann, F. Werner, and B. Lohmann, "A tube-mpc approach to autonomous multi-vehicle racing on high-speed ovals," *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 1, pp. 368–378, 2022.

[5] A. Muraleedharan, I. Simões, and G. Pau, "Randomized model predictive control for autonomous racing," in *2025 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2025, pp. 1078–1085.

[6] "A2RL autonomous car race," https://a2rl.io/autonomous-car-race, accessed: 2025-01-18.

[7] N. Musiu, E. Mascaro, A. Raji, A. De Felice, S. Sorrentino, and M. Bertogna, "A comprehensive benchmark of vehicle dynamics models for autonomous racing: a deep dive into mpc," *Vehicle System Dynamics*, pp. 1–33, 2026.

[8] N. A. Spielberg, M. Brown, and J. C. Gerdes, "Neural network model predictive motion control applied to automated driving with unknown friction," *IEEE Transactions on Control Systems Technology*, vol. 30, no. 5, pp. 1934–1945, 2021.

[9] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll, "Real-time neural mpc: Deep learning model predictive control for quadrotors and agile robotic platforms," *IEEE Robotics and Automation Letters*, vol. 8, no. 4, pp. 2397–2404, 2023.

[10] X. Zhang, M. Bujarbaruah, and F. Borrelli, "Near-optimal rapid mpc using neural networks: A primal-dual policy learning framework," *IEEE Transactions on Control Systems Technology*, vol. 29, no. 5, pp. 2102–2114, 2020.

[11] T. Weiss and M. Behl, "Deepracing: A framework for autonomous racing," in *2020 Design, automation & test in Europe conference & exhibition (DATE)*. IEEE, 2020, pp. 1163–1168.

[12] P. Drews, G. Williams, B. Goldfain, E. A. Theodorou, and J. M. Rehg, "Aggressive deep driving: Combining convolutional neural networks and model predictive control," in *Conference on Robot Learning*. PMLR, 2017, pp. 133–142.

[13] P. R. Wurman, S. Barrett, K. Kawamoto, J. MacGlashan, K. Subramanian, T. J. Walsh, R. Capobianco, A. Devlic, F. Eckert, F. Fuchs *et al.*, "Outracing champion gran turismo drivers with deep reinforcement learning," *Nature*, vol. 602, no. 7896, pp. 223–228, 2022.

[14] M. Jaritz, R. De Charette, M. Toromanoff, E. Perot, and F. Nashashibi, "End-to-end race driving with deep reinforcement learning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 2070–2075.

[15] M. Vasco, T. Seno, K. Kawamoto, K. Subramanian, P. R. Wurman, and P. Stone, "A super-human vision-based reinforcement learning agent for autonomous racing in gran turismo," *arXiv preprint arXiv:2406.12563*, 2024.

[16] A. Draeger, S. Engell, and H. Ranke, "Model predictive control using neural networks," *IEEE Control Systems Magazine*, vol. 15, no. 5, pp. 61–66, 1995.

[17] Z. Yan and J. Wang, "Model predictive control of nonlinear systems with unmodeled dynamics based on feedforward and recurrent neural networks," *IEEE Transactions on Industrial Informatics*, vol. 8, no. 4, pp. 746–756, 2012.

[18] P. Kumar, J. B. Rawlings, and S. J. Wright, "Industrial, large-scale model predictive control with structured neural networks," *Computers & chemical engineering*, vol. 150, p. 107291, 2021.

[19] J. Drgoňa, D. Picard, M. Kvasnica, and L. Helsen, "Approximate model predictive building control via machine learning," *Applied energy*, vol. 218, pp. 199–216, 2018.

[20] A. Marchetti, B. Chachuat, and D. Bonvin, "Modifier-adaptation methodology for real-time optimization," *Industrial & engineering chemistry research*, vol. 48, no. 13, pp. 6022–6033, 2009.

[21] K. Seel, A. B. Kordabad, S. Gros, and J. T. Gravdahl, "Convex neural network-based cost modifications for learning model predictive control," *IEEE Open Journal of Control Systems*, vol. 1, pp. 366–379, 2022.

[22] S. Hirt, M. Pfefferkorn, and R. Findeisen, "Safe and stable closed-loop learning for neural-network-supported model predictive control," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*. IEEE, 2024, pp. 4764–4770.

[23] S. Hirt, A. Höhl, J. Pohlodek, J. Schaeffer, M. Pfefferkorn, R. D. Braatz, and R. Findeisen, "Safe learning-based optimization of model predictive control: Application to battery fast-charging," in *2025 American Control Conference (ACC)*. IEEE, 2025, pp. 2817–2822.

[24] D. Casanova, "On minimum time vehicle manoeuvring: The theoretical optimal lap," Ph.D. dissertation, Cranfield University, Cranfield, UK, 2000.

[25] J. L. Vazquez, M. Bruhlmeier, A. Liniger, A. Rupenyan, and J. Lygeros, "Optimization-based hierarchical motion planning for autonomous racing," in *IEEE International Conference on Intelligent Robots and Systems*, 2020.

[26] H. B. Pacejka and E. Bakker, "The magic formula tyre model," *Vehicle System Dynamics*, vol. 21, 1992.

[27] A. Muraleedharan, H. Okuda, and T. Suzuki, "Improvement of control performance of sampling based model predictive control using gpu," in *IEEE Intelligent Vehicles Symposium (IV)*, 2019, pp. 1999–2004.

[28] A. Muraleedharan, H. Okuda, and T. Suzuki, "Real-time implementation of randomized model predictive control for autonomous driving," *IEEE Transactions on Intelligent Vehicles*, vol. 7, no. 1, pp. 11–20, 2022.

[29] A. Muraleedharan, "Realization of safe autonomous driving using randomized model predictive control," Ph.D. dissertation, Nagoya University, 2023.

[30] A. Muraleedharan, A.-T. Tran, H. Okuda, and T. Suzuki, "Scenario-based model predictive speed controller considering probabilistic constraint for driving scene with pedestrian," in *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, 2020, pp. 1–7.

[31] A.-T. Tran, A. Muraleedharan, H. Okuda, and T. Suzuki, "Scenario-based stochastic mpc for vehicle speed control considering the interaction with pedestrians," in *20th World Congress of the International Federation of Automatic Control*, 2020.

[32] A. Muraleedharan, H. Okuda, and T. Suzuki, "Path tracking control using model predictive control with on gpu implementation for autonomous driving," *Journal of Arid Land Studies*, vol. 28, no. S, pp. 163–167, 2018.

[33] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[34] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.